

Decroly
SANTANDER

# UD3

## API REST CON SPRING BOOT

MP_0613
Desarrollo Web en Entorno Servidor

## 3.3 La capa Service y la configuración

## Qué vas a aprender en este tema

Hasta el momento, tu `ProductoRestController` hace demasiadas cosas: recibe peticiones, guarda los datos, valida y decide qué errores lanzar. En este tema vas a poner cada cosa en su sitio. Crearás la **capa de servicios**, donde vive la lógica de negocio de la tienda, y aprenderás el mecanismo con el que Spring conecta unas capas con otras: la **inyección de dependencias**. También aprenderás a sacar la configuración del código con propiedades y perfiles, a ahorrarte código repetitivo con **Lombok** y a escribir tus primeras **pruebas unitarias**.

## 1. Por qué una capa de servicios

### El punto de partida

Repasa la última versión de `ProductoRestController`. En una sola clase conviven cuatro tareas muy distintas:

| Tarea | Código que la hace | ¿Es trabajo de un controlador? |
|---|---|---|
| Atender peticiones HTTP y construir respuestas | Métodos con `@GetMapping`, `ResponseEntity`… | Sí |
| Guardar los datos | `List<Producto> productos`, `AtomicLong secuencia` | No: es trabajo de la capa de datos |
| Aplicar reglas de negocio | `validar()`: nombre obligatorio y precio no negativo | No: es trabajo de la capa de servicios |
| Decidir qué pasa si algo no existe | `buscarOLanzar()` | No: es trabajo de la capa de servicios |

¿Qué problema hay? Imagina que la tienda quiere una segunda forma de crear productos, por ejemplo importándolos desde un fichero. Las reglas de validación están atrapadas dentro del controlador REST: habría que copiarlas. Y cada copia es una oportunidad de que una se actualice y la otra no.

### El principio de responsabilidad única

El **principio de responsabilidad única** (la "S" de los principios SOLID) dice que **una clase debe tener un único motivo para cambiar**. Nuestro controlador tiene al menos tres: que cambie la API, que cambie la forma de guardar los datos o que cambien las reglas del negocio. La arquitectura por capas aplica este principio a nivel de aplicación:

| Capa | Su único motivo para cambiar | En este tema |
|---|---|---|
| Controller | Cambia la forma de comunicarse con el exterior (rutas, formato, códigos). | Queda reducido: recibe, delega y responde. |
| Service | Cambian las reglas del negocio. | Nueva capa: validaciones, cálculos, errores de negocio. |
| Repository | Cambia la forma de guardar los datos. | De momento, los datos siguen en una lista dentro del servicio. Más adelante pasarán a un repositorio real. |

## Programar contra interfaces

Una **interfaz** de Java declara **qué** se puede hacer, sin decir **cómo**. Es un contrato: cualquier clase que la implemente se compromete a ofrecer esos métodos.

**El contrato del servicio de productos**

```
public interface ProductoService {
    List<Producto> listar();
    Producto buscarPorId(Long id);
    Producto crear(Producto producto);
    Producto modificar(Long id, Producto datos);
    void borrar(Long id);
}
```

Hoy la implementación guardará los productos en una lista; más adelante en MySQL. Si el controlador solo conoce la interfaz, ese cambio no le afectará en absoluto. A esto se le llama **programar contra interfaces**, no contra implementaciones.

## El acoplamiento

Dos clases están **acopladas** cuando un cambio en una obliga a cambiar la otra. Mira esta forma de usar el servicio desde el controlador:

**Acoplamiento alto**

```
public class ProductoRestController {
    private final ProductoService servicio = new ProductoServiceMemoria();
}
```

Aunque el atributo es de tipo interfaz, el controlador sigue eligiendo con `new` **qué implementación concreta** usa. Para cambiar a MySQL habría que modificar el controlador; y para probarlo con datos falsos, también. La solución es que el controlador **no cree** su servicio, sino que **se lo den hecho**. Eso es exactamente lo que hace el contenedor de Spring.

## 2. Inyección de dependencias

### Dependencias y quién las crea

Una **dependencia** es un objeto que otro necesita para trabajar: el controlador depende del servicio; más adelante, el servicio dependerá del repositorio. La **inyección de dependencias** consiste en que el objeto **no crea** sus dependencias, sino que las **recibe** desde fuera. En Spring, quien las crea y las entrega es el contenedor:

1. Al arrancar, el contenedor escanea el paquete raíz y encuentra las clases marcadas con anotaciones de componente.
2. Crea un objeto de cada una (un **bean**).
3. Mira qué necesita cada bean para construirse y le entrega los beans adecuados.

Esta idea es la **inversión de control** que conociste, aplicada a las dependencias.

### Los estereotipos: marcar las clases como componentes

Para que el contenedor cree un bean de una clase, hay que marcarla. Spring tiene una anotación general y tres especializadas, llamadas **estereotipos**, que indican a qué capa pertenece:

| Anotación | Capa | Particularidad |
|---|---|---|
| `@Component` | Cualquiera | La anotación general. Las otras tres la incluyen. |
| `@Controller` / `@RestController` | Controller | Registra además las rutas de sus métodos. |
| `@Service` | Service | Solo documenta que es lógica de negocio. Técnicamente, igual que `@Component`. |
| `@Repository` | Repository | Traduce los errores de la base de datos a excepciones de Spring. |

## Tres formas de inyectar

**Las tres formas de inyectar una dependencia**

```
// 1) Por atributo
@RestController
public class ProductoRestController {
    @Autowired
    private ProductoService productoService;
}

// 2) Por setter
@RestController
public class ProductoRestController {
    private ProductoService productoService;

    @Autowired
    public void setProductoService(ProductoService productoService) {
        this.productoService = productoService;
    }
}

// 3) Por constructor (la recomendada)
@RestController
public class ProductoRestController {
    private final ProductoService productoService;

    public ProductoRestController(ProductoService productoService) {
        this.productoService = productoService;
    }
}
```

| | Por atributo | Por setter | Por constructor |
|---|---|---|---|
| ¿Se ve qué necesita la clase? | Solo leyendo sus atributos | Solo leyendo sus métodos | Sí: está en la firma del constructor |
| ¿Puede ser `final`? | No | No | Sí: no se puede cambiar por error |
| ¿Se puede crear con `new` en una prueba? | No sin trucos (atributo privado) | Sí, llamando al setter | Sí: `new ProductoRestController(servicioFalso)` |
| ¿Necesita `@Autowired`? | Sí | Sí | No, si la clase tiene un solo constructor |

Por eso Spring recomienda la **inyección por constructor**, y es la que usaremos. La inyección por atributo está muy extendida en tutoriales y en código antiguo, así que debes saber reconocerla.

### Los beans son únicos

Cuando arranca la aplicación, Spring crea **un solo objeto** de cada bean (cada clase con `@Service`, `@RestController`…). Ese mismo objeto atiende **todas** las peticiones que llegan, de todos los usuarios. A esto se le llama ámbito (*scope*) **singleton**.

Además, el servidor no atiende las peticiones de una en una. Si Ana y Luis piden algo a la vez, cada petición se atiende en un **hilo** distinto, al mismo tiempo, y los dos hilos usan **el mismo objeto**.

> **PARA ENTENDER · La pizarra y el cuaderno**
>
> Un bean es como la pizarra del aula: solo hay una y la usa todo el mundo. Si Ana escribe un número en la pizarra para hacer una cuenta y, mientras tanto, Luis lo borra y escribe el suyo, Ana termina la cuenta con el número de Luis. En cambio, lo que cada uno apunta en **su cuaderno** no lo toca nadie más.
>
> En Java, la pizarra es un **atributo** del bean y el cuaderno es una **variable local** del método.

Este servicio suma los precios de varios productos. Tiene un fallo escondido:

**Lo que NO hay que hacer**

```
@Service
public class ResumenService {

    private final ProductoService productoService;  // BIEN: es una dependencia
    private double total;                           // MAL: dato de una peticion

    public ResumenService(ProductoService productoService) {
        this.productoService = productoService;
    }

    public double sumarPrecios(List<Long> ids) {
        total = 0;
        for (Long id : ids) {
            total = total + productoService.buscarPorId(id).getPrecio();
        }
        return total;
    }
}
```

Esto es lo que puede pasar si Ana y Luis lo usan a la vez:

| Momento | Hilo de Ana | Hilo de Luis | Valor de total |
|---|---|---|---|
| 1 | `total = 0` | | 0 |
| 2 | Suma un producto de 20 € | | 20 |
| 3 | | `total = 0` | **0** (borra lo de Ana) |

| Momento | Hilo de Ana | Hilo de Luis | Valor de total |
|---|---|---|---|
| 4 | Suma un producto de 30 € | | 30 |
| 5 | Devuelve `total` | | Ana recibe **30 €**, pero lo correcto era **50 €** |

El error no aparece siempre, solo cuando dos peticiones coinciden. Por eso es tan difícil de detectar: en tus pruebas todo funciona y, con muchos usuarios, falla de vez en cuando.

La solución es usar una **variable local**. Cada vez que se llama al método se crea un `total` nuevo, solo para esa llamada:

**La versión correcta**

```
public double sumarPrecios(List<Long> ids) {
    double total = 0;          // cada llamada tiene el suyo
    for (Long id : ids) {
        total = total + productoService.buscarPorId(id).getPrecio();
    }
    return total;
}
```

**Qué puede guardar un bean en sus atributos**

- **Sus dependencias**: otros beans que recibe por el constructor, normalmente `final`.
- **Datos comunes a todos que no cambian**, como un valor leído de `application.properties`.

Todo lo que es de **una petición concreta** (el usuario que la hace, un resultado intermedio, un contador…) va en **parámetros y variables locales**.

### Inicialización diferida: @Lazy

Los beans se crean todos al arrancar. Si un bean es muy costoso de crear y casi nunca se usa, se puede marcar con `@Lazy` para que se cree la **primera vez que se necesita**. Tiene un inconveniente: si su creación falla, no te enterarás al arrancar, sino en mitad del funcionamiento. Por eso se usa poco.

## Práctica 1. El servicio de productos

1. Crea el paquete `es.decroly.tienda.services` y dentro la interfaz `ProductoService` con los cinco métodos del apartado 1.
2. En el mismo paquete, crea la implementación. Fíjate en que contiene exactamente lo que sobraba en el controlador:

**ProductoServiceMemoria.java**

```
package es.decroly.tienda.services;

import es.decroly.tienda.domain.Producto;
import es.decroly.tienda.exceptions.PeticionIncorrectaException;
import es.decroly.tienda.exceptions.RecursoNoEncontradoException;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class ProductoServiceMemoria implements ProductoService {

    private final List<Producto> productos = new ArrayList<>();
    private final AtomicLong secuencia = new AtomicLong();

    public ProductoServiceMemoria() {
        anadir("Teclado mecanico", 49.90, 15);
        anadir("Raton inalambrico", 19.95, 40);
        anadir("Monitor 24 pulgadas", 139.00, 8);
    }

    @Override
    public List<Producto> listar() {
        return productos;
    }

    @Override
    public Producto buscarPorId(Long id) {
        for (Producto p : productos) {
            if (p.getId().equals(id)) {
                return p;
            }
        }
        throw new RecursoNoEncontradoException(
                "No existe el producto " + id);
    }

    @Override
    public Producto crear(Producto producto) {
        validar(producto);
        producto.setId(secuencia.incrementAndGet());
        productos.add(producto);
        return producto;
    }

    @Override
    public Producto modificar(Long id, Producto datos) {
        validar(datos);
        Producto producto = buscarPorId(id);
        producto.setNombre(datos.getNombre());
        producto.setPrecio(datos.getPrecio());
        producto.setStock(datos.getStock());
        return producto;
    }
```

```
    @Override
    public void borrar(Long id) {
        Producto producto = buscarPorId(id);
        productos.remove(producto);
    }

    private void anadir(String nombre, double precio, int stock) {
        Long id = secuencia.incrementAndGet();
        productos.add(new Producto(id, nombre, precio, stock));
    }

    private void validar(Producto p) {
        if (p.getNombre() == null || p.getNombre().isBlank()) {
            throw new PeticionIncorrectaException("Nombre obligatorio");
        }
        if (p.getPrecio() < 0) {
            throw new PeticionIncorrectaException("Precio negativo");
        }
    }
}
```

1. Reescribe el controlador. Ya no tiene datos ni reglas: recibe el servicio por constructor, delega en él y construye la respuesta HTTP.

**ProductoRestController.java (versión 5)**

```
package es.decroly.tienda.controllers;

import es.decroly.tienda.domain.Producto;
import es.decroly.tienda.services.ProductoService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/productos")
public class ProductoRestController {

    private final ProductoService productoService;

    public ProductoRestController(ProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping
    public List<Producto> listar() {
        return productoService.listar();
    }

    @GetMapping("/{id}")
    public Producto buscarPorId(@PathVariable Long id) {
        return productoService.buscarPorId(id);
```

```
    }

    @PostMapping
    public ResponseEntity<Producto> crear(@RequestBody Producto producto) {
        Producto creado = productoService.crear(producto);
        URI direccion = URI.create("/api/productos/" + creado.getId());
        return ResponseEntity.created(direccion).body(creado);
    }

    @PutMapping("/{id}")
    public Producto modificar(@PathVariable Long id,
                              @RequestBody Producto datos) {
        return productoService.modificar(id, datos);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> borrar(@PathVariable Long id) {
        productoService.borrar(id);
        return ResponseEntity.noContent().build();
    }
}
```

`import org.springframework.web.bind.annotation.*;` importa de golpe todas las anotaciones de ese paquete. Es habitual en controladores, que usan muchas.

1. Ejecuta toda la colección de Postman. Todo debe funcionar exactamente igual que antes, incluidos los errores: las excepciones que lanza el servicio llegan al manejador global igual que si las lanzara el controlador.
2. Compara el tamaño del controlador con la versión del tema 3.2. ¿Qué responsabilidades le quedan?

**Experimenta con el contenedor**

- Añade al constructor de `ProductoServiceMemoria` la línea `System.out.println("Creando ProductoServiceMemoria");`. Arranca: ¿cuándo aparece el mensaje? Haz diez peticiones: ¿cuántas veces aparece? ¿Por qué?
- Quita `@Service` de `ProductoServiceMemoria` y arranca. Lee el error: ¿qué dice que falta?
- Vuelve a poner `@Service`. Añade `@Lazy` a la clase y también delante del parámetro del constructor del controlador (`@Lazy ProductoService productoService`). ¿Cuándo aparece ahora el mensaje?
- Deja el código como estaba (sin `@Lazy` ni el `println`).

## 3. Varias implementaciones de un servicio

La ventaja de programar contra interfaces es que puede haber **varias implementaciones** del mismo contrato. Pero entonces surge un problema: si hay dos beans de tipo `ProductoService`,

¿cuál inyecta Spring en el controlador? Si no se lo indicas, **no arranca** y te dice que esperaba un bean y ha encontrado dos. Hay tres formas de resolverlo:

| Mecanismo | Dónde se pone | Qué hace | Cuándo usarlo |
|---|---|---|---|
| `@Primary` | En una de las implementaciones | La marca como la preferida cuando hay varias. | Hay una implementación "normal" y otras para casos especiales. |
| `@Qualifier("nombre")` | En el punto de inyección | Pide un bean concreto por su nombre. | Una clase concreta necesita una implementación concreta. |
| `@ConditionalOnProperty` | En las implementaciones | Solo crea el bean si una propiedad tiene cierto valor. Solo existe una implementación. | Elegir la implementación con configuración, sin tocar el código. |

Por defecto, el nombre de un bean es el de su clase empezando en minúscula: `productoServiceMemoria`, `productoServiceJson`.

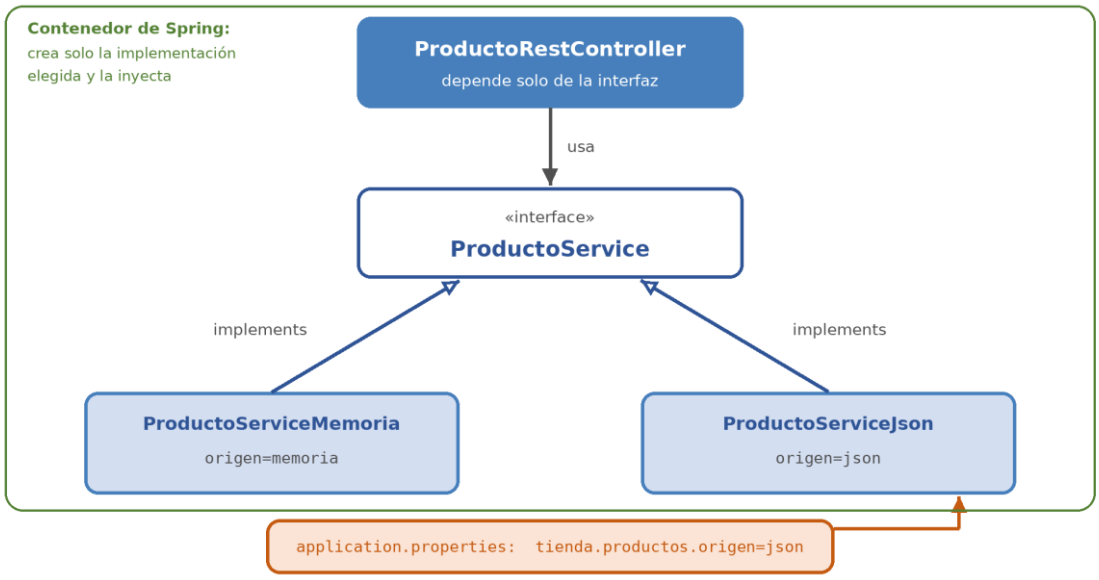



*Figura 1. El controlador solo conoce la interfaz; una propiedad decide qué implementación crea el contenedor.*

**El patrón Strategy**

Tener varias implementaciones intercambiables de una misma interfaz, y elegir una sin que su usuario lo note, es un **patrón de diseño** llamado *Strategy* (estrategia). Spring lo facilita tanto que lo usarás constantemente sin darte cuenta: cambiar de base de datos será cambiar de implementación.

### Evitar código repetido: una clase base

Nuestra segunda implementación leerá los productos iniciales de un fichero JSON, pero el resto de operaciones (buscar, crear, validar…) serán idénticas. Copiar la clase sería un error. La solución de la programación orientada a objetos es la **herencia**: una **clase abstracta** con todo lo común, y dos subclases que solo cambian de dónde salen los datos iniciales.

## Práctica 2. Productos desde un fichero JSON

1. Renombra `ProductoServiceMemoria` a `ProductoServiceBase` (clic derecho › **Refactor › Rename**, Mayús+F6), hazla `abstract`, quítale `@Service` y cambia su constructor para que reciba la lista inicial:

**ProductoServiceBase.java (lo que cambia)**

```
public abstract class ProductoServiceBase implements ProductoService {

    private final List<Producto> productos;
    private final AtomicLong secuencia;

    protected ProductoServiceBase(List<Producto> iniciales) {
        this.productos = new ArrayList<>(iniciales);
        long maxId = 0;                       // mayor id de los iniciales
        for (Producto p : iniciales) {
            if (p.getId() > maxId) {
                maxId = p.getId();
            }
        }
        this.secuencia = new AtomicLong(maxId);
    }

    // listar, buscarPorId, crear, modificar, borrar, validar: igual
    // (borra el metodo anadir, ya no hace falta)
}
```

La secuencia empieza en el mayor id de los datos iniciales, para que el siguiente producto creado no repita un id existente.

1. Crea de nuevo `ProductoServiceMemoria`, ahora como subclase que solo aporta sus datos:

**ProductoServiceMemoria.java**

```
package es.decroly.tienda.services;

import es.decroly.tienda.domain.Producto;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class ProductoServiceMemoria extends ProductoServiceBase {

    public ProductoServiceMemoria() {
        super(List.of(
                new Producto(1L, "Teclado mecanico", 49.90, 15),
                new Producto(2L, "Raton inalambrico", 19.95, 40),
                new Producto(3L, "Monitor 24 pulgadas", 139.00, 8)));
    }
}
```

1. En `src/main/resources`, crea el fichero `productos.json`:

**productos.json**

```
[
  { "id": 1, "nombre": "Portatil 15", "precio": 649.00, "stock": 5 },
  { "id": 2, "nombre": "Auriculares BT", "precio": 59.90, "stock": 25 },
  { "id": 3, "nombre": "Disco SSD 1 TB", "precio": 89.95, "stock": 12 },
  { "id": 4, "nombre": "Hub USB-C", "precio": 29.50, "stock": 30 }
]
```

1. Crea la segunda implementación. Recibe por constructor el `ObjectMapper` de Jackson, otro bean que Spring Boot ya tiene configurado, y lo usa para leer el fichero:

**ProductoServiceJson.java**

```
package es.decroly.tienda.services;

import es.decroly.tienda.domain.Producto;
import org.springframework.core.io.ClassPathResource;
import org.springframework.stereotype.Service;
import tools.jackson.databind.ObjectMapper;

import java.io.IOException;
import java.io.InputStream;
import java.util.List;

@Service
public class ProductoServiceJson extends ProductoServiceBase {

    public ProductoServiceJson(ObjectMapper mapper) throws IOException {
        super(cargar(mapper));
    }

    private static List<Producto> cargar(ObjectMapper mapper)
            throws IOException {
        ClassPathResource fichero =
                new ClassPathResource("productos.json");
        try (InputStream entrada = fichero.getInputStream()) {
            return List.of(mapper.readValue(entrada, Producto[].class));
        }
    }
}
```

| Elemento | Explicación |
|---|---|
| `tools.jackson.databind.ObjectMapper` | Paquete de Jackson 3 (Spring Boot 4). Con Spring Boot 3 sería `com.fasterxml.jackson.databind.ObjectMapper`. |
| `ClassPathResource` | Localiza un fichero dentro de `src/main/resources`, que al empaquetar queda dentro del jar. |
| `readValue(entrada, Producto[].class)` | Deserializa el array JSON en un array de `Producto`. Es la misma conversión que hace Spring con `@RequestBody`. |
| `super(cargar(mapper))` | La llamada a `super` debe ser la primera instrucción; por eso la lectura va en un método estático auxiliar. |

1. Arranca la aplicación. Falla. Lee el mensaje de error y localiza la frase que indica cuántos beans de tipo `ProductoService` ha encontrado.
2. **Con @Primary:** añade `@Primary` a `ProductoServiceJson`. Arranca y lista los productos: ¿de dónde salen? Quita `@Primary`.

3. **Con @Qualifier:** en el constructor del controlador, escribe `@Qualifier("productoServiceMemoria") ProductoService productoService`. Arranca y comprueba. Quita `@Qualifier`.
4. **Con @ConditionalOnProperty:** añade estas anotaciones a las dos implementaciones:

**Selección por configuración**

```
// En ProductoServiceMemoria
@Service
@ConditionalOnProperty(name = "tienda.productos.origen",
                       havingValue = "memoria", matchIfMissing = true)

// En ProductoServiceJson
@Service
@ConditionalOnProperty(name = "tienda.productos.origen",
                       havingValue = "json")
```

`matchIfMissing = true` hace que la implementación en memoria sea la que se use si la propiedad no está definida.

1. Añade `tienda.productos.origen=json` a `application.properties`, arranca y lista. Cámbiala a `memoria` y repite. **No has tocado ni una línea de Java** para cambiar de fuente de datos.
2. Ejecuta toda la colección de Postman con las dos configuraciones.

# 4. Configuración de la aplicación

## Por qué sacar la configuración del código

Hay valores que no son lógica, sino **configuración**: el porcentaje de IVA, el importe a partir del cual el envío es gratis, la dirección de la base de datos, el puerto. Si están escritos en el código Java, cambiarlos obliga a recompilar. Si están en la configuración:

- Se cambian sin tocar el código: el mismo `.jar` sirve para el ordenador del programador, el servidor de pruebas y el de producción.
- Los datos sensibles, como las contraseñas, no quedan escritos en el código ni en Git.
- Todos los valores ajustables están reunidos en un sitio.

## Propiedades y YAML

En `application.properties`, cada línea es un par `clave=valor`, y las claves se organizan con puntos. Spring Boot admite también el formato **YAML** (`application.yml`), que usa sangría en

lugar de repetir los prefijos. Son equivalentes; en este curso seguiremos con `.properties`, pero debes reconocer los dos:

**application.properties**

```
spring.application.name=tienda-decroly
tienda.nombre=Tienda Decroly
tienda.iva=21
tienda.envio-gratis-desde=50
tienda.productos.origen=memoria
```

**Lo mismo en application.yml**

```yaml
spring:
  application:
    name: tienda-decroly
tienda:
  nombre: Tienda Decroly
  iva: 21
  envio-gratis-desde: 50
  productos:
    origen: memoria
```

### Leer propiedades: @Value y @ConfigurationProperties

| | `@Value("${clave}")` | `@ConfigurationProperties` |
|---|---|---|
| Qué lee | Una propiedad suelta | Un grupo de propiedades con el mismo prefijo |
| Cómo se usa | Sobre un parámetro del constructor o un atributo | Sobre una clase o un `record` cuyos campos son las propiedades |
| Valor por defecto | `@Value("${tienda.nombre:Mi tienda}")` | El valor inicial del campo |
| Cuándo usarlo | Uno o dos valores aislados | Configuración propia de la aplicación (lo habitual) |

`@ConfigurationProperties` aplica **enlace flexible**: la propiedad `tienda.envio-gratis-desde` rellena el campo `envioGratisDesde`. También convierte los tipos: el texto `21` del fichero llega como `double`.

### Perfiles

Un **perfil** es un nombre para un entorno de ejecución: `dev` (desarrollo), `test` (pruebas), `prod` (producción)… Además de `application.properties`, puedes crear ficheros `application-{perfil}.properties`. Cuando un perfil está activo, sus propiedades se añaden a las generales y, si coinciden, **las sustituyen**.

| Fichero | Cuándo se aplica |
|---|---|
| `application.properties` | Siempre. |
| `application-dev.properties` | Solo con el perfil `dev` activo. |
| `application-prod.properties` | Solo con el perfil `prod` activo. |

El perfil se activa con la propiedad `spring.profiles.active`, que se puede indicar de varias formas:

| Dónde | Cómo |
|---|---|
| En `application.properties` | `spring.profiles.active=dev` |
| En IntelliJ | **Run › Edit Configurations**: en Ultimate, campo **Active profiles**; en cualquier edición, **Program arguments** `--spring.profiles.active=prod` |
| Con Maven | `mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=prod` |
| Al ejecutar el jar | `java -jar tienda-decroly.jar --spring.profiles.active=prod` |
| Variable de entorno | `SPRING_PROFILES_ACTIVE=prod` |

### ¿Qué valor gana?

Una misma propiedad puede estar definida en varios sitios. Spring Boot aplica un orden de prioridad; de mayor a menor, estas son las fuentes más habituales:

1. Argumentos de la línea de comandos (`--tienda.iva=10`).
2. Variables de entorno del sistema (`TIENDA_IVA=10`: mayúsculas y guiones bajos en lugar de puntos).
3. `application-{perfil}.properties` del perfil activo.
4. `application.properties`.

Así es como se despliega en la práctica: el fichero trae valores razonables y el servidor de producción los sobrescribe con variables de entorno, por ejemplo la contraseña de la base de datos.

## Práctica 3. Configuración propia de la tienda

1. Añade a `application.properties` las propiedades `tienda.nombre`, `tienda.iva` y `tienda.envio-gratis-desde` del ejemplo anterior.
2. Crea el paquete `es.decroly.tienda.config` y dentro este `record`:

**TiendaProperties.java**

```
package es.decroly.tienda.config;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "tienda")
public record TiendaProperties(String nombre,
                               double iva,
                               double envioGratisDesde) {
}
```

1. Añade `@ConfigurationPropertiesScan` a la clase principal, junto a `@SpringBootApplication`, para que Spring busque y cree los beans de configuración.
2. Crea en el paquete `dto` el `record` que usaremos como respuesta. Un **DTO** (*Data Transfer Object*) es un objeto pensado solo para enviar datos, distinto de las clases del negocio:

**PrecioDto.java**

```
package es.decroly.tienda.dto;

public record PrecioDto(Long idProducto, double base,
                        double iva, double total) {
}
```

1. Crea en `services` un segundo servicio que calcula precios. Fíjate en que **un servicio depende de otro servicio y de la configuración**, y recibe los dos por constructor:

**PrecioService.java**

```
package es.decroly.tienda.services;

import es.decroly.tienda.config.TiendaProperties;
import es.decroly.tienda.domain.Producto;
import es.decroly.tienda.dto.PrecioDto;
import org.springframework.stereotype.Service;

@Service
public class PrecioService {

    private final ProductoService productoService;
    private final TiendaProperties tienda;

    public PrecioService(ProductoService productoService,
                         TiendaProperties tienda) {
        this.productoService = productoService;
        this.tienda = tienda;
    }

    public PrecioDto calcular(Long idProducto) {
        Producto producto = productoService.buscarPorId(idProducto);
        double base = producto.getPrecio();
        double total = base * (1 + tienda.iva() / 100);
        total = Math.round(total * 100) / 100.0;   // redondeo a centimos
        return new PrecioDto(idProducto, base, tienda.iva(), total);
    }
}
```

**PARA ENTENDER · ¿Todo servicio necesita una interfaz?**

`PrecioService` es una clase sin interfaz. La interfaz merece la pena cuando se prevén **varias implementaciones** (como `ProductoService`, que tendrá una versión con base de datos). Para un servicio que solo tendrá una forma de hacer las cosas, una clase es suficiente. Programar contra interfaces es una herramienta, no una obligación.

Recuerda también el error de la división entera del tema 3.1: aquí `iva()` es `double`, así que `iva() / 100` da 0,21.

1. Añade `PrecioService` como segunda dependencia del controlador de productos (un segundo parámetro en el constructor) y este endpoint:

**ProductoRestController.java (nuevo endpoint)**

```
@GetMapping("/{id}/precio")
public PrecioDto precio(@PathVariable Long id) {
    return precioService.calcular(id);
}
```

1. Prueba `GET /api/productos/3/precio`. Cambia `tienda.iva` a 10, reinicia y repite.
2. Crea `application-dev.properties` con `tienda.nombre=Tienda Decroly (DESARROLLO)` y `logging.level.es.decroly.tienda=DEBUG`.

3. Crea `application-prod.properties` con `tienda.productos.origen=json`.
4. Añade `spring.profiles.active=dev` a `application.properties`. Arranca y localiza en el log la línea que indica el perfil activo.
5. Arranca con el perfil `prod` desde IntelliJ (**Program arguments**: `--spring.profiles.active=prod`). ¿De dónde salen ahora los productos?
6. Con el perfil `prod`, añade además el argumento `--tienda.iva=4` y consulta un precio. ¿Qué fuente de configuración ha ganado?

## 5. Lombok

### Qué problema resuelve

La clase `Producto` tiene 4 atributos y más de 30 líneas: casi todo son getters, setters y constructores que no aportan nada. **Lombok** es una librería que **genera ese código automáticamente al compilar**, a partir de anotaciones. El fichero `.java` queda corto, pero el `.class` compilado contiene todos los métodos, como si los hubieras escrito.

| Anotación | Genera |
|---|---|
| `@Getter` / `@Setter` | Getters / setters de todos los atributos (o de uno, si se pone sobre el atributo). |
| `@NoArgsConstructor` | Constructor sin parámetros. |
| `@AllArgsConstructor` | Constructor con todos los atributos, en el orden en que están declarados. |
| `@RequiredArgsConstructor` | Constructor con los atributos `final`. **Perfecto para la inyección por constructor.** |
| `@ToString` | Método `toString()` con todos los atributos. |
| `@EqualsAndHashCode` | Métodos `equals()` y `hashCode()` basados en los atributos. |
| `@Data` | Todo junto: getters, setters, `toString`, `equals`, `hashCode` y constructor de atributos `final`. |
| `@Slf4j` | El atributo `log` del logger, sin escribir `LoggerFactory.getLogger(…)`. |

**ATENCIÓN · Cuidado con @Data**

`@Data` es cómoda, pero más adelante nuestras clases del dominio serán entidades de base de datos con relaciones entre ellas (un producto pertenece a una familia, una familia tiene productos). Con `@Data`, el `toString()` y el `equals()` generados recorrerían esas relaciones y podrían entrar en un bucle infinito. Por eso en el dominio usaremos `@Getter` y `@Setter` en su lugar.

### Ventajas e inconvenientes

| Ventajas | Inconvenientes |
|---|---|
| Clases mucho más cortas y legibles | El código que se ejecuta no está a la vista |
| Al añadir un atributo, sus métodos se generan solos | Necesita soporte en el IDE y en el compilador |
| Muy extendida en proyectos Spring | Es una dependencia más del proyecto |

## Práctica 4. Simplifica el código con Lombok

1. Añade la dependencia a `pom.xml` (sin versión: la gestiona Spring Boot) y recarga Maven. En un proyecto nuevo, basta con marcar **Lombok** en Spring Initializr.

**pom.xml**

```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

**ATENCIÓN · Si al compilar no encuentra los getters**

Desde Java 23, el compilador no ejecuta automáticamente los procesadores de anotaciones como Lombok. Si al compilar aparecen errores del tipo *cannot find symbol: method getNombre()*, añade dentro de `<build><plugins>` esta configuración (Spring Initializr ya la genera cuando marcas Lombok):

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <configuration>
        <annotationProcessorPaths>
            <path>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
            </path>
        </annotationProcessorPaths>
    </configuration>
</plugin>
```

En IntelliJ, si aparece el aviso **Enable annotation processing**, acéptalo.

1. Simplifica `Producto`:

**Producto.java con Lombok**

```
package es.decroly.tienda.domain;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Producto {

    private Long id;
    private String nombre;
    private double precio;
    private int stock;
}
```

1. Abre el panel **Structure** de IntelliJ (Alt+7) con `Producto.java` abierto: verás los métodos generados aunque no estén en el fichero.
2. En el controlador y en `PrecioService`, borra el constructor y añade `@RequiredArgsConstructor` a la clase. Los atributos `final` deben seguir ahí:

**ProductoRestController.java con Lombok**

```
@RestController
@RequestMapping("/api/productos")
@RequiredArgsConstructor
public class ProductoRestController {

    private final ProductoService productoService;
    private final PrecioService precioService;

    // ... endpoints sin cambios, sin constructor ...
}
```

1. Añade `@Slf4j` a `ProductoServiceBase` y registra con `log.info("Producto creado: {}", producto.getId());` cada creación. Crea un producto y comprueba el log.
2. Ejecuta de nuevo toda la colección de Postman: el comportamiento no debe haber cambiado.

## 6. Probar y documentar el código

### Por qué escribir pruebas automáticas

Hasta ahora has comprobado tu código a mano, con Postman. Funciona, pero cada vez que cambias algo tendrías que repetir todas las pruebas, y nadie lo hace. Una **prueba automática** es código que ejecuta tu código y comprueba que el resultado es el esperado. Se ejecutan todas en segundos, cada vez que quieras, y avisan si un cambio ha estropeado algo que funcionaba.

| Tipo de prueba | Qué comprueba | Velocidad | En este bloque |
|---|---|---|---|
| **Unitaria** | Una clase aislada, sin servidor ni base de datos | Milisegundos | Este tema: los servicios |
| **De integración** | Que varias piezas funcionan juntas (controlador + servicio, servicio + base de datos) | Segundos | Temas 4.3 y 5.2 |
| **De extremo a extremo** | La aplicación completa, como un usuario | Minutos | Lo que haces a mano con Postman |

La capa de servicios es la más fácil de probar: no sabe nada de HTTP y, gracias a la inyección por constructor, se puede crear con `new` pasándole las dependencias que queramos.

## JUnit 5

**JUnit** es la librería de pruebas estándar en Java. Viene incluida en `spring-boot-starter-test`. Una prueba es un método anotado con `@Test` y suele seguir tres pasos (el patrón **Preparar – Actuar – Comprobar**):

**Anatomía de una prueba**

```
@Test
void crearAsignaIdYLoAnade() {
    // Preparar: crear los datos de partida
    Producto nuevo = new Producto(null, "Webcam", 35.0, 10);

    // Actuar: ejecutar lo que se quiere probar
    Producto creado = servicio.crear(nuevo);

    // Comprobar: verificar el resultado
    assertNotNull(creado.getId());
    assertEquals(4, servicio.listar().size());
}
```

| Elemento | Para qué sirve |
|---|---|
| `@Test` | Marca un método como prueba. |
| `@BeforeEach` | Método que se ejecuta **antes de cada** prueba. Sirve para partir siempre de un estado limpio. |
| `assertEquals(esperado, real)` | Falla si los valores no coinciden. Con decimales: `assertEquals(168.19, real, 0.001)`, con un margen de error. |
| `assertTrue(condicion)` / `assertNotNull(x)` | Falla si la condición es falsa o el valor es `null`. |
| `fail("mensaje")` | Hace fallar la prueba en ese punto. Sirve para probar los casos de error (ver el recuadro siguiente). |

**Cómo probar que se lanza una excepción**

Para comprobar un caso de error, llama al método dentro de un `try`. Justo después de la llamada, pon un `fail`:

```
try {
    servicio.buscarPorId(99L);
    fail("Deberia haber lanzado RecursoNoEncontradoException");
} catch (RecursoNoEncontradoException ex) {
    // correcto: es la excepcion que esperabamos
}
```

- Si el método **lanza** la excepción, la ejecución salta al `catch`, el `fail` no llega a ejecutarse y la prueba pasa.
- Si el método **no la lanza**, se ejecuta el `fail` y la prueba falla con ese mensaje.

JUnit tiene también `assertThrows`, que hace lo mismo en una línea, pero necesita una lambda. Lo verás en proyectos reales.

Cada prueba debe ser **independiente**: no puede depender de que otra se haya ejecutado antes. Su nombre debe decir qué comprueba, para que, cuando falle, se sepa qué se ha roto sin leer el código.

## Javadoc

**Javadoc** es el formato estándar para documentar código Java. Se escribe en comentarios que empiezan por `/**` justo antes de una clase o un método. IntelliJ lo muestra al pasar el ratón sobre cualquier uso del método, y se puede generar con él una web de documentación. Lo más útil es documentar las **interfaces**, porque son el contrato que usan las demás clases:

**Javadoc en ProductoService**

```
/**
 * Operaciones de negocio sobre los productos de la tienda.
 */
public interface ProductoService {

    /**
     * Busca un producto por su identificador.
     *
     * @param id identificador del producto
     * @return el producto encontrado
     * @throws RecursoNoEncontradoException si no existe ese producto
     */
    Producto buscarPorId(Long id);
}
```

| Etiqueta | Uso |
|---|---|
| `@param nombre descripción` | Describe un parámetro. |
| `@return descripción` | Describe lo que devuelve el método. |

| Etiqueta | Uso |
|---|---|
| `@throws Excepcion descripción` | Indica cuándo lanza una excepción. |

## Práctica 5. Pruebas unitarias y Javadoc

1. En `src/test/java/es/decroly/tienda`, crea el paquete `services` y dentro la clase de prueba (por convenio, el nombre de la clase probada seguido de `Test`):

**ProductoServiceMemoriaTest.java**

```
package es.decroly.tienda.services;

import es.decroly.tienda.domain.Producto;
import es.decroly.tienda.exceptions.PeticionIncorrectaException;
import es.decroly.tienda.exceptions.RecursoNoEncontradoException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class ProductoServiceMemoriaTest {

    private ProductoService servicio;

    @BeforeEach
    void preparar() {
        servicio = new ProductoServiceMemoria();
    }

    @Test
    void listarDevuelveLosProductosIniciales() {
        assertEquals(3, servicio.listar().size());
    }

    @Test
    void buscarPorIdExistenteDevuelveElProducto() {
        Producto producto = servicio.buscarPorId(1L);
        assertEquals("Teclado mecanico", producto.getNombre());
    }

    @Test
    void buscarPorIdInexistenteLanzaExcepcion() {
        try {
            servicio.buscarPorId(99L);
            fail("Deberia haber lanzado RecursoNoEncontradoException");
        } catch (RecursoNoEncontradoException ex) {
            // correcto
        }
    }

    @Test
    void crearAsignaIdYLoAnade() {
```

```
        Producto nuevo = new Producto(null, "Webcam", 35.0, 10);
        Producto creado = servicio.crear(nuevo);
        assertNotNull(creado.getId());
        assertEquals(4, servicio.listar().size());
    }

    @Test
    void crearConPrecioNegativoLanzaExcepcion() {
        Producto malo = new Producto(null, "Webcam", -5.0, 10);
        try {
            servicio.crear(malo);
            fail("Deberia haber lanzado PeticionIncorrectaException");
        } catch (PeticionIncorrectaException ex) {
            // correcto
        }
    }

    @Test
    void borrarEliminaElProducto() {
        servicio.borrar(1L);
        try {
            servicio.buscarPorId(1L);
            fail("El producto 1 deberia haberse borrado");
        } catch (RecursoNoEncontradoException ex) {
            // correcto
        }
    }
}
```

1. Ejecuta las pruebas con el triángulo verde junto al nombre de la clase. El panel inferior mostrará cada prueba en verde (superada) o rojo (fallida).
2. Provoca un fallo: en `validar()`, cambia `p.getPrecio() < 0` por `p.getPrecio() < -10`. Ejecuta las pruebas, lee el mensaje de la que falla y deshaz el cambio.
3. Crea `PrecioServiceTest`. Observa que, gracias a la inyección por constructor, se construye el servicio con `new` y una configuración inventada, sin arrancar Spring:

**PrecioServiceTest.java**

```
class PrecioServiceTest {

    @Test
    void calcularAplicaElIvaConfigurado() {
        TiendaProperties config = new TiendaProperties("Prueba", 21, 50);
        PrecioService precios =
                new PrecioService(new ProductoServiceMemoria(), config);

        PrecioDto precio = precios.calcular(3L);   // monitor de 139 EUR

        assertEquals(168.19, precio.total(), 0.001);
    }
}
```

1. Añade una segunda prueba que compruebe el cálculo con un IVA del 10 %, y una tercera que compruebe que pedir el precio de un producto inexistente lanza `RecursoNoEncontradoException`.
1. Ejecuta todas las pruebas del proyecto desde la terminal con `mvnw.cmd test` y localiza el resumen final (*Tests run*, *Failures*).
2. Documenta con Javadoc los cinco métodos de `ProductoService`. Pasa el ratón sobre una llamada a `buscarPorId` en el controlador y comprueba que IntelliJ muestra tu documentación.
3. Genera la documentación con **Tools › Generate JavaDoc** y abre el `index.html` resultante en el navegador.